Mathematical Foundations of Big Data Engineering

Comprehensive Synthesis Exercise

Sessions 1–4: Distributed Systems, Kafka, Spark, Data Lakes

Answer Key

(Statement + concise solutions)

Big Data Engineering – XICS404

Academic Year 2025–2026

Document Overview

This document presents every question of the synthesis exercise together with a **concise answer**: the key formula, the substitution, and the numerical result. No pedagogical commentary is included; see the Professor Version for full explanations.

Convention. Each answer box contains: (1) the governing formula in a display, (2) the numerical substitution, and (3) the boxed result.

Scenario: The *GlobalCart* Data Platform

GlobalCart – global e-commerce platform, 25 countries. Three event streams: *Clickstream*, *Orders* (must not be lost), *Telemetry*. Stack: Web/Mobile apps → Kafka → Spark Structured Streaming → S3 Data Lake (Parquet) → BI & Analytics.

1. Part I – Ingestion Layer: Kafka Cluster Sizing

Setup. $N = 7$ brokers, $p = 0.01$ failure/hour per broker. `orders` topic: $P = 16$ partitions, replication factor $r = 3$, `min.insync.replicas`$= 2$.

1.1 Q1 – Hash partitioning and load balancing

$n = 4\,800\,000$ orders/hour, $P = 16$ partitions, uniform `customer_id`.

(a) **Expected load and standard deviation per partition.**

Answer

$$\mathbb{E}[X_i] = \frac{n}{P} = \frac{4\,800\,000}{16} = \boxed{300\,000 \text{ orders/partition}}$$

$$\sigma(X_i) = \sqrt{n \cdot \tfrac{1}{P}(1 - \tfrac{1}{P})} = \sqrt{4.8 \times 10^6 \cdot \tfrac{1}{16} \cdot \tfrac{15}{16}} = \sqrt{281\,250} \approx \boxed{530}$$

(b) **Coefficient of variation.**

Answer

$$\frac{\sigma(X_i)}{\mathbb{E}[X_i]} = \frac{530}{300\,000} \approx \boxed{0.18\%}$$

Load imbalance is negligible – hash partitioning is well-balanced at this scale.

(c) **Black Friday: 25% of orders from one buyer.**

Answer

$$\text{Hot-partition load} = 0.25 \times n + \frac{0.75\,n}{P} = 1\,200\,000 + 225\,000 = \boxed{1\,425\,000} \approx 4.75 \times \text{average}$$

Consequence: hot partition – backpressure, consumer-lag spike, cannot be fixed by scaling.

1.2 Q2 – Replication and reliability

(d) **Probability of data loss (all 3 replicas down).**

Answer

$$\mathbb{P}(\text{loss}) = p^r = (0.01)^3 = \boxed{10^{-6}}$$

(e) **Probability partition becomes read-only (exactly 2 replicas down).**

Answer

$$\mathbb{P}(K = 2) = \binom{3}{2} p^2 (1 - p) = 3 \times (0.01)^2 \times 0.99 = \boxed{2.97 \times 10^{-4}}$$

(f) **Union bound: probability at least one of $P = 16$ partitions is lost.**

Answer

$$\mathbb{P}(\geq 1 \text{ lost}) \leq P \cdot p^r = 16 \times 10^{-6} = \boxed{1.6 \times 10^{-5}}$$

$\approx$ one incident per 7 years of continuous operation.

1.3 Q3 – Quorums and consistency

Cassandra cluster: $N_c = 5$ replicas.

(g) **Strong-consistency inequality.**

Answer

$$\boxed{R + W > N_c} \quad \Longleftrightarrow \quad R + W \geq 6$$

(h) **All (R, W) pairs with $1 \leq R, W \leq 5$ satisfying strong consistency.**

Answer

15 pairs: $(1,5), (2,4), (2,5), (3,3), (3,4), (3,5), (4,2), (4,3), (4,4), (4,5), (5,1), (5,2), (5,3), (5,4), (5,5)$.

(i) **Pair minimising $\max(R, W)$.**

Answer

$$\boxed{(R, W) = (3,3)}, \quad \max = 3 \quad (\text{balanced quorum: } \lceil N_c/2 \rceil + 1)$$

Symmetric read/write latency; tolerates 2 replica failures.

(j) **Under a 2-replica network partition.**

Answer

3 replicas reachable. Reads available iff $R \leq 3$; writes iff $W \leq 3$. $(3,3)$ remains fully operational – AP behaviour. Configurations with $W > 3$ sacrifice write availability for consistency (CP).

1.4 Q4 – Little's Law and in-flight events

$\lambda = 50\,000$ events/s, $L = 40$ ms.

(k) **Average events in flight.**

Answer

$$\overline{N} = \lambda \cdot L = 50\,000 \times 0.040 = \boxed{2\,000 \text{ events}}$$

(l) **Memory budget at 2 KB per event.**

Answer

$$M = 2\,000 \times 2 \text{ KB} = \boxed{4 \text{ MB}} \quad (\text{negligible vs. 16 GB broker RAM})$$

2. Part II – Producer & Consumer Reliability

2.1 Q5 – Producer acknowledgements

$p = 0.005$, $r = 3$.

(a) **Loss probabilities.**

Answer

Configuration	$\mathbb{P}(\text{loss})$
`acks=1`	5×10^{-3}
`acks=all`, $m = 2$	$(0.005)^2 = 2.5 \times 10^{-5}$
`acks=all`, $m = 3$	$(0.005)^3 = 1.25 \times 10^{-7}$

(b) **Expected lost orders per hour ($\lambda = 50\,000/\text{s} \Rightarrow 1.8 \times 10^8/\text{h}$).**

Answer

Configuration	Lost orders/hour	
`acks=1`	9.0×10^5	*catastrophic*
`acks=all`, $m = 2$	4 500	*unacceptable (financial)*
`acks=all`, $m = 3$	22.5	*marginal; needs retries*

2.2 Q6 – Retries (geometric model)

$q = 0.03$, $L_0 = 8$ ms, $B = 100$ ms, $R = 4$ retries (5 total attempts).

(c) **Expected number of attempts.**

Answer

$$\mathbb{E}[T] = \frac{1}{1-q} = \frac{1}{0.97} \approx \boxed{1.031}$$

(d) **Expected end-to-end latency.**

Answer

$$\mathbb{E}[\text{latency}] = \mathbb{E}[T] \cdot L_0 + (\mathbb{E}[T] - 1) \cdot B = 1.031 \times 8 + 0.031 \times 100 \approx \boxed{11.3 \text{ ms}}$$

(e) **Residual loss probability after 5 attempts.**

Answer

$$q^{R+1} = (0.03)^5 = \boxed{2.43 \times 10^{-8}}$$

(f) **Overall loss combining Q5(a) (`acks=all`, $m = 2$) and Q6(c).**

Answer

$$\varepsilon_{\text{tot}} \approx 2.5 \times 10^{-5} + 2.43 \times 10^{-8} \approx \boxed{2.50 \times 10^{-5}}$$

The retry term is $10^3\times$ smaller; broker failure dominates.

2.3 Q7 – Delivery semantics

(g) **The 2×2 classification table.**

Answer

	No duplicates	**Possible duplicates**
No loss	Exactly-once	At-least-once
Possible loss	At-most-once	(useless)

(h) **Expected processings per record ($\alpha = 2 \times 10^{-4}$).**

Answer

$$\mathbb{E}[\#\text{proc}] = 1 + \alpha = 1.0002$$

(i) **Duplicate processings per hour.**

Answer

$$\alpha \cdot \lambda \cdot 3\,600 = 2 \times 10^{-4} \times 50\,000 \times 3\,600 = \boxed{36\,000 \text{ duplicates/hour}}$$

(j) **Why exactly-once is required.**

Answer

36 000 duplicate processings/hour on a non-idempotent SQL debit $\Rightarrow$ 36 000 double-charges/hour. Exactly-once sets $\mathbb{E}[\#\text{proc}] = 1$, eliminating duplicates entirely.

2.4 Q8 – Consumer-group capacity and lag

$P = 16$, $\mu = 5\,000$ rec/s per consumer.

(k) **Stability check with $C = 8$ consumers.**

Answer

$$C_{\text{active}} \cdot \mu = 8 \times 5\,000 = 40\,000 \text{ rec/s} < \lambda = 50\,000 \text{ rec/s} \Rightarrow \textbf{NOT stable}$$

Lag grows at 10 000 rec/s.

(l) **Lag after 60-s spike at $\lambda' = 100\,000$ rec/s (still $C = 8$).**

Answer

$$N_0 = (\lambda' - C_{\text{active}}\mu) \times T_{\text{spike}} = (100\,000 - 40\,000) \times 60 = \boxed{3.6 \times 10^6 \text{ records}}$$

(m) **Catch-up time after spike (requires $C \geq 11$).**

Answer

With $C = 11$ ($C_{\text{active}}\mu = 55\,000$):

$$T_{\text{catch-up}} = \frac{N_0}{C_{\text{active}}\mu - \lambda} = \frac{3.6 \times 10^6}{55\,000 - 50\,000} = \boxed{720\text{ s} = 12\text{ min}}$$

(n) **Maximum useful consumers.**

Answer

$$C_{\max} = P = \boxed{16} \quad \text{(extra consumers stay idle)}$$

(o) **Minimum partitions for $\lambda^\star = 120\,000$ rec/s.**

Answer

$$P^\star = \left\lceil \frac{\lambda^\star}{\mu} \right\rceil = \left\lceil \frac{120\,000}{5\,000} \right\rceil = \boxed{24\text{ partitions}}$$

3. Part III – Processing Layer: Spark Streaming

3.1 Q9 – Spark parallelism and Amdahl's Law

$E = 25$ executors, $k = 4$ cores, $P_0 = 4\,800$ blocks, $t_{\text{task}} = 8$ s, $f = 0.92$.

(a) **Total parallel slots.**

Answer

$S = E \cdot k = 25 \times 4 = \boxed{100}$

(b) **Number of waves.**

Answer

$\lceil P_0/S \rceil = \lceil 4800/100 \rceil = \boxed{48}$ waves

(c) **Stage time.**

Answer

$T_{\text{stage}} = 48 \times 8 = \boxed{384 \text{ s}}$

(d) **Coalesce to $P_1 = 200$; stage time and concern.**

Answer

$t'_{\text{task}} = 8 \times (4800/200) = 192$ s; $\lceil 200/100 \rceil = 2$ waves; $T'_{\text{stage}} = 2 \times 192 = \boxed{384 \text{ s}}$ (same).
Per-task size: 600 GB/200 = 3 GB – dangerously close to the 4 GB per-slot heap limit.

(e) **Amdahl speed-up at $S = 100$; theoretical max.**

Answer

$$\text{Speed-up}(100) = \frac{1}{0.08 + 0.92/100} = \frac{1}{0.0892} \approx \boxed{11.2} \qquad \text{Max} = \frac{1}{1-f} = \frac{1}{0.08} = \boxed{12.5}$$

Already at 90% of ceiling; serial fraction dominates.

(f) **Doubling to $S' = 200$.**

Answer

$$\text{Speed-up}(200) = \frac{1}{0.08 + 0.92/200} \approx \boxed{11.8} \quad \Rightarrow \quad \text{improvement} = \frac{11.8 - 11.2}{11.2} \approx \boxed{5.4\%}$$

Not cost-effective; reduce serial fraction instead.

3.2 Q10 – Windowing and watermarks

$w = 15$ min, $s = 3$ min, $\theta = 5$ min, $K = 25$ countries, 32 bytes/aggregate, $\mathbb{E}[D] = 60$ s.

(g) **Windows per event.**

Answer

$k = w/s = 15/3 = \boxed{5}$

(h) **Open windows at any time.**

Answer

$N_{\text{open}} = \lceil (w+\theta)/s \rceil = \lceil 20/3 \rceil = \boxed{7}$

(i) **Total state memory.**

Answer

$M = K \times N_{\text{open}} \times 32 = 25 \times 7 \times 32 = \boxed{5\,600 \text{ bytes} \approx 5.5 \text{ KB}}$

(j) **Tumbling window state reduction.**

Answer

$N'_{\text{open}} = \lceil 20/15 \rceil = 2$; $M' = 25 \times 2 \times 32 = 1\,600$ bytes. Factor: $5\,600/1\,600 = \boxed{3.5\times}$ smaller.

(k) **Drop probability with $\theta = 5$ min.**

Answer

$\mu = 1/60 \text{ s}^{-1}$, $\theta = 300$ s:

$$\mathbb{P}(\text{drop}) = e^{-\mu\theta} = e^{-5} \approx \boxed{6.74 \times 10^{-3}}$$

(l) **Watermark $\theta^\star$ for drop rate $\leq 10^{-4}$.**

Answer

$$\theta^\star = \frac{1}{\mu} \ln\left(\frac{1}{\alpha}\right) = 60 \times \ln(10\,000) = 60 \times 9.21 \approx \boxed{553 \text{ s} \approx 9.2 \text{ min}}$$

3.3 Q11 – Batch-vs-streaming cost crossover

$C_b = \$12/\text{run}$, $r_s = \$0.30/\text{min}$.

(m) **Crossover refresh interval.**

Answer

$$T^\star = \frac{C_b}{r_s} = \frac{12}{0.30} = \boxed{40 \text{ min}}$$

(n) **Cheaper option at 10-min freshness.**

Answer

10 min < 40 min $\Rightarrow$ streaming wins.
Streaming: $60 \times \$0.30 = \$18/\text{h}$. Batch every 10 min: $6 \times \$12 = \$72/\text{h}$. **Saving: $54/hour.**

(o) **Monthly saving (30 days, 24/7).**

Answer

$\$54 \times 720 = \boxed{\$38\,880/\text{month}}$

4. Part IV – Storage Layer: Data Lake

Setup. $V = 4$ TB, $n = 40$ columns, $W = 480$ bytes/row, $N = 8.3 \times 10^9$ rows, row groups $G = 100\,000$.

4.1 Q12 – Column pruning and compression

CSV+gzip $r_{\text{row}} = 3$; Parquet+Snappy $r_{\text{col}} = 6$; $|S| = 6$ columns.

(a) **Column-pruning ratio.**

Answer

$\rho_{\text{prune}} = 1 - 6/40 = \boxed{85\%}$

(b) **Bytes read – CSV compressed.**

Answer

$B_{\text{CSV}} = V/r_{\text{row}} = 4/3 \approx \boxed{1.33 \text{ TB}}$

(c) **Bytes read – Parquet with column pruning.**

Answer

$B_{\text{Parquet}}^{\text{col}} = 0.15 \times 4/6 = \boxed{100 \text{ GB}}$

(d) **Combined speed-up Parquet vs. CSV.**

Answer

$\text{Speed-up} = 1.33 \text{ TB}/100 \text{ GB} = \boxed{13.3\times}$

4.2 Q13 – Predicate pushdown

$\sigma = 1/25 = 0.04$, $G = 100\,000$.

(e) **Row-group skip probability (unsorted data).**

Answer

$\mathbb{P}(\text{skip}) = (1 - 0.04)^{100\,000} = 0.96^{100\,000} \approx \boxed{0}$
Nothing is skipped – pushdown is useless on unsorted data.

(f) **Total number of row groups.**

Answer

$R = N/G = 8.3 \times 10^9/10^5 = \boxed{83\,000}$

(g) **Fraction read after sorting by `country`.**

Answer

Contiguous matching rows $\Rightarrow$ fraction $\approx \sigma = \boxed{4\%}$ of row groups.

(h) **Bytes read – Parquet, column pruning + sorted pushdown.**

Answer

$B = \sigma \times B^{\text{col}}_{\text{Parquet}} = 0.04 \times 100\ \text{GB} = \boxed{4\ \text{GB}}$ (333× less than CSV).

4.3 Q14 – Partition design and pruning

$c_1 = 25,\ c_2 = 3,\ c_3 = 12,\ c_4 = 31.$

(i) **Total leaf directories.**

Answer

$D = 25 \times 3 \times 12 \times 31 = \boxed{27\,900}$

(j) **Partition-pruning ratio (France, 2024, 7 days).**

Answer

$$\rho_{\text{read}} = \tfrac{1}{25} \times \tfrac{1}{3} \times \tfrac{1}{12} \times \tfrac{7}{31} = \frac{7}{27\,900} \approx \boxed{2.51 \times 10^{-4}}$$

Only 0.025% of directories touched.

(k) **Overall scan fraction (partition × column × pushdown).**

Answer

$$\rho_{\text{eff}} = 2.51 \times 10^{-4} \times 0.15 \times 0.04 \approx \boxed{1.5 \times 10^{-6}}$$

Roughly one part per million of the raw dataset.

4.4 Q15 – The small-file problem

$\tau = 100$ ms, $\beta = 200$ MB/s, 250 files of 8 MB per day-leaf.

(l) **Optimal average file size.**

Answer

$s_{\text{opt}} = \tau \cdot \beta = 0.1 \times 200 = \boxed{20\ \text{MB}}$
Current 8 MB files are *below* this threshold.

(m) **Files read for France, 7 days.**

Answer

$F = 7 \times 250 = \boxed{1\,750\ \text{files}}$

(n) **Total read time – current layout.**

Answer

$B = 7 \times 2\ \text{GB} = 14\ \text{GB}$

$$T_{\text{read}} = 1\,750 \times 0.1 + 14\,000/200 = 175 + 70 = \boxed{245\ \text{s}}$$

Metadata overhead: $175/245 = 71\%$ of total time.

(o) **Read time after compaction (1 file of 2 GB per leaf).**

Answer

$T' = 7 \times 0.1 + 14\,000/200 = 0.7 + 70 = \boxed{70.7 \text{ s}}$
Speed-up: $245/70.7 \approx \boxed{3.5\times}$

4.5 Q16 – The high-cardinality trap

Adding `user_id` (5×10^6) between `country` and `year`.

(p) **New total leaf directories.**

Answer

$D' = 25 \times 5 \times 10^6 \times 3 \times 12 \times 31 \approx \boxed{1.4 \times 10^{11} \text{ directories}}$

(q) **Average file size per leaf.**

Answer

$\bar{s} = 4 \text{ TB}/1.4 \times 10^{11} \approx \boxed{28.7 \text{ bytes/file}}$

(r) **Mathematical rejection.**

Answer

28.7 bytes $\ll s_{\text{opt}} = 20$ MB: metadata overhead is $\sim 10^7\times$ larger than I/O time. Additionally, 1.4×10^{11} directory entries collapse the metastore. High-cardinality columns belong *inside* the file (clustering/Z-ordering), **not** as partition columns.

5. Part V – End-to-End Synthesis

5.1 Q17 – Combined throughput budget

$\lambda = 50\,000$, $P = 16$, $\mu = 5\,000$.

(a) **Maximum sustainable throughput and bottleneck.**

Answer

$\lambda_{\max} = P \cdot \mu = 16 \times 5\,000 = \boxed{80\,000 \text{ rec/s}}$
Bottleneck: partition count caps C_{active}.

(b) **Plan for +60% growth ($\lambda_{\text{new}} = 80\,000 \rightarrow$ plan for $96\,000$).**

Answer

$P^\star = \lceil 96\,000/5\,000 \rceil = \boxed{20}$ partitions, $C^\star = 20$ consumers. Extra cost: $4 \times \$0.05 = \$0.20/\text{min} \approx \$144/\text{day}, \approx \$4\,320/\text{month}$.

5.2 Q18 – End-to-end reliability budget

Kafka $\varepsilon_K = 2.50 \times 10^{-5}$, Spark $\varepsilon_S = 10^{-6}$, S3 $\varepsilon_{S3} = 10^{-7}$.

(c) **End-to-end loss probability.**

Answer

$\varepsilon_{\text{tot}} \approx 2.50 \times 10^{-5} + 10^{-6} + 10^{-7} \approx \boxed{2.61 \times 10^{-5}}$ (Kafka dominates)

(d) **Expected lost orders per year; five-nines check.**

Answer

Orders/year: $50\,000 \times 3600 \times 24 \times 365 \approx 1.58 \times 10^{12}$.
Lost: $1.58 \times 10^{12} \times 2.61 \times 10^{-5} \approx \boxed{4.1 \times 10^{7}}$ orders/year.
Five-nines requires $\varepsilon \leq 10^{-5}$: currently $2.6\times$ over budget.

5.3 Q19 – End-to-end latency budget

Kafka 40 ms, Spark $\Delta t = 5$ s, S3 200 ms.

(e) **Total end-to-end latency.**

Answer

$L_{\text{tot}} = 40 + 5\,000 + 200 = \boxed{5\,240 \text{ ms} \approx 5.2 \text{ s}}$

(f) **Sub-second target: dominant stage and fix.**

Answer

Spark accounts for $5\,000/5\,240 \approx 95\%$ of latency. Fix: reduce trigger interval Δt to ≤ 500 ms, or migrate to Spark Continuous Processing / Apache Flink.

5.4 Q20 – End-to-end cost and storage

$V_{\text{daily}} = 12$ GB/day, $p = \$0.023$/GB-month.

(g) **Monthly storage cost (1 year accumulated data).**

Answer

$V_{\text{year}} = 12 \times 365 = 4\,380$ GB uncompressed.

Format	r	Compressed	Monthly cost
CSV+gzip	3	1 460 GB	\$33.6
Parquet+Snappy	6	730 GB	\$16.8
Parquet+Zstd	10	438 GB	\$10.1

(h) **Investment priority.**

Answer

1. **S3 access patterns** (sorting + compaction):
   333× I/O reduction (Q13) and 3.5× read-time reduction (Q15) – essentially free.
2. **Kafka right-sizing** to match real demand (∼ \$4 320/month for +4 consumers, Q17(b)).
3. **Spark serial-fraction reduction** (Amdahl ceiling at 12.5×, only 5% gain from doubling cluster, Q9(f)).
4. Storage TCO (\$23 saved/month CSV→Zstd) is negligible but format choice drives the access cost – use Parquet+Zstd.

End of Answer Key